4. OOP Scenario-Based Problems

Concepts Required:
- Object-Oriented Programming
- Abstraction (abstract classes / interfaces)
- final keyword (variables, methods, classes)
- static variables and methods

Instructions:
• Use proper OOP design
• Avoid procedural coding
• Follow encapsulation strictly
• Write clean, modular code

Problem 1: Payment Gateway System

Design an abstract class Payment with abstract method processPayment(). Implement CreditCardPayment and UPIPayment classes. Use static variable to track total transactions and final method to print receipt.

Problem 2: Bank Interest Calculation Engine

Create an abstract class BankAccount with abstract calculateInterest(). Implement SavingsAccount and FixedDepositAccount. Interest rate should be final and shared using static variable.

Problem 3: Ride Booking Platform

Design an abstract class Ride with calculateFare(). Implement BikeRide and CabRide. Use static baseFare and final surgeMultiplier logic.

Problem 4: Employee Attendance System

Create abstract Employee with markAttendance(). Implement FullTimeEmployee and ContractEmployee. Use static companyName and final employeeId.

Problem 5: Online Examination System

Create abstract Exam with evaluate(). Implement ObjectiveExam and CodingExam. Total marks should be final and exam count tracked using static variable.

Problem 6: E-Commerce Discount Engine

Create abstract class DiscountPolicy with applyDiscount(). Implement SeasonalDiscount and LoyaltyDiscount. Discount limits should be final and static counters used for analytics.

Problem 7: Vehicle Toll Management

Design abstract Vehicle with calculateToll(). Implement Car, Truck, Bike. Use static tollRate and final vehicleNumber.

Problem 8: Library Management System

Create abstract class Book with issueBook(). Implement ReferenceBook and LendingBook. Library capacity should be static and ISBN final.

Problem 9: Online Food Delivery Platform

Design abstract class Order with calculateBill(). Implement VegOrder and NonVegOrder. Use static taxRate and final orderId.

Problem 10: Insurance Premium Calculator

Create abstract InsurancePolicy with calculatePremium(). Implement HealthInsurance and VehicleInsurance. Base premium static and policyNumber final.